MODULE 05 · DOMAIN 1 - DESIGN APPLICATIONS

Agent Bricks: configure before you code

Knowledge Assistant, Supervisor Agent and Information Extraction: what each builds for you, and when to write your own.

SECTION 1 of 6 EXAM WEIGHT 14% MODULE 05 of 40

admin@sthithapragna.com

Flat-pack or bespoke joinery? If a ready-made builder fits the job, configure it. Write code only for what it cannot do.

What you provide, and what you get back

BUILDER	YOU PROVIDE	YOU GET
Knowledge Assistant	Documents or an AI Search index, a description, instructions	A chat endpoint that answers with citations
Supervisor Agent	Subagents: Genie Agents, agent endpoints, UC functions, MCP servers	One endpoint that delegates and combines results
Information Extraction	Documents or text, and a schema of fields	Structured fields, at scale, in a table
Custom Agents	Your own code, wrapped as a ResponsesAgent	Anything, with the most effort

ON THE EXAM

The guide says Multiagent Supervisor. The docs now say Supervisor Agent. Same product.

From a folder of documents to a cited chatbot

01
Point
At a volume of files, or an AI Search index.

02
Describe
What the agent is for, plus instructions.

03
Build
Databricks parses, chunks, indexes and tunes.

04
Test
Answers with citations, in the Playground.

05
Improve
Experts add questions and guidelines.

ON THE EXAM

Every rag step from module one is still happening. Databricks does it for you, and keeps it tunable by non-coders.

The details worth knowing

ASPECT	DETAIL
Sources	Volume files: txt, pdf, md, ppt, pptx, doc, docx. Or a UC table with a file column
Existing index	An AI Search index built with GTE large, BGE large or Qwen3 embeddings
Output	An agent endpoint. Answers carry citations to the source documents
Improving it	Expert questions and guidelines, importable from and exportable to UC tables
Requirements	Unity Catalog and serverless compute

ON THE EXAM

Citations are the headline feature. When a scenario says users must see where an answer came from, Knowledge Assistant is a strong candidate.

Knowledge Assistant or a custom rag agent?

KNOWLEDGE ASSISTANT WHEN

The job is cited Q and A

- Questions over documents
- Citations required
- Experts tune it without code
- Speed to production matters

CUSTOM RAG AGENT WHEN

The job needs control

- Special chunking or retrieval logic
- Actions or tools beyond answering
- Unsupported sources or models
- Exact control of every prompt

THE TIEBREAKER

If a scenario asks for Q and A over documents, fast, with citations or no-code tuning, choose Knowledge Assistant. Reach for code only when a stated need rules it out.

One front door, several specialists behind it

01
Request
A user asks a mixed question.

02
Plan
The supervisor works out what is needed.

03
Delegate
Genie for tables, an assistant for documents.

04
Act
A UC function or MCP tool, if needed.

05
Combine
One answer, drawn from every result.

ON THE EXAM

The supervisor adds coordination, not knowledge. Every fact still comes from a subagent.

The details worth knowing

ASPECT	DETAIL
Subagents	Genie Agents, agent endpoints, UC functions, MCP servers, custom agents
Output	One agent endpoint that delegates and combines results
Improving it	Expert feedback in natural language improves coordination
Data access	Subagents still enforce it. Genie uses each user's own permissions
Also supports	Long-running tasks, and adding subagents later via the SDK

ON THE EXAM

Supervisor Agent is not the Supervisor API, a separate beta that ends on 30 September 2026.

Supervisor Agent or a custom orchestrator?

SUPERVISOR AGENT WHEN

Coordinating existing pieces

- Genie Agents and assistants already exist
- Routing can be described in words
- Experts should tune coordination
- One endpoint is wanted quickly

CUSTOM ORCHESTRATOR WHEN

Routing needs code

- Fixed branching the supervisor cannot express
- Proprietary logic between steps
- A specific agent framework is required
- Behaviour the builder does not support

THE TIEBREAKER

Databricks' own docs say: try Supervisor Agent before building a custom orchestrator on Databricks Apps.

One engine, three ways to use it

JOB	AGENT BRICKS UI	SQL FUNCTION
Turn PDFs, DOCX, images, slides into text and tables	Document Parsing	ai_parse_document
Pull named fields out, using your schema	Information Extraction	ai_extract
Assign one of your labels, up to 500 or more	Classification	ai_classify

ON THE EXAM

The UI and the SQL function share the same engine. Design in the UI, then run at scale in SQL or through the REST API.

Extract fields, or answer questions?

INFORMATION EXTRACTION WHEN

The output is a table

- Same fields from every document
- Thousands of documents
- Results feed reports or systems
- No conversation needed

KNOWLEDGE ASSISTANT WHEN

The output is an answer

- Different question each time
- A person is asking
- Answer needs citations
- Conversation is the interface

THE TIEBREAKER

Same fields from every document means extraction. A different question from every user means an assistant.

Which brick?

01

Q and A over docs?

Knowledge Assistant.

›

02

Fields from docs?

Information Extraction, then ai_extract.

›

03

Several sources or agents?

Supervisor Agent.

›

04

Still no fit?

Custom Agents, in code.

ON THE EXAM

Walk the list in order and stop at the first fit. Databricks' guidance, and the exam, both reward the simplest option that works.

> An HR team has 3,000 policy PDFs in a Unity Catalog volume. Staff want to ask questions and see which policy each answer came from.
>
> It must be live in two weeks, and HR specialists, who do not code, must be able to improve the answers.

What should the engineer use?

A. A custom LangGraph rag agent with hand-tuned chunking and retrieval

B. A model fine-tuned on the policy PDFs, retrained whenever they change

C. A Knowledge Assistant over the volume, improved with expert guidelines

D. A Genie Agent pointed at the policy PDFs in the Unity Catalog volume

◆ PAUSE AND DECIDE

What should the engineer use?

A Custom LangGraph rag agent

Would work, but slower to build and HR cannot tune it without code.

B Fine-tuned model

No citations, and every policy change needs a retrain.

C Knowledge Assistant

Cited answers, fast to build, experts improve it without code.

D Genie Agent

Genie works over structured tables only. It cannot read PDFs.

THE SIGNAL

See which policy, two weeks, and specialists who do not code. Three signals, one builder.

> A sales operations team already has a Genie Agent over its sales tables and a Knowledge Assistant over its contract templates.
>
> Managers want one assistant that can answer both kinds of question in the same conversation.

What is the most efficient way to deliver this?

A. A Supervisor Agent with the Genie Agent and the Knowledge Assistant as subagents

B. Export the sales tables as CSV files and add them to the Knowledge Assistant

C. Write a custom multi-agent orchestrator on Databricks Apps from scratch

D. Give managers links to both existing assistants

◆ PAUSE AND DECIDE

What is the most efficient way to deliver this?

A Supervisor Agent over both

Coordinates existing agents into one endpoint. No rebuilding.

B Sales CSVs into the assistant

Tables become stale, fuzzy text. Loses Genie's exact SQL answers.

C Custom orchestrator from scratch

Works, but docs say try Supervisor Agent first. Nothing needs code.

D Two separate links

Fails the one-conversation requirement.

THE SIGNAL

Already has a Genie Agent and a Knowledge Assistant, and wants one conversation. Existing pieces plus coordination means Supervisor Agent.

A procurement team holds 80,000 supplier contracts as PDFs. It needs renewal date, liability cap and governing law for each one, in a table.

The team wants to design and check the fields on a few sample contracts first, then process new contracts every night.

Which approach fits best?

A. A Knowledge Assistant over the contracts, queried contract by contract

B. Design in Information Extraction, then run ai_extract on a schedule

C. A Supervisor Agent with a separate subagent for each field needed

D. Run ai_summarize over each contract and store the summaries in a table

◆ PAUSE AND DECIDE

Which approach fits best?

A Knowledge Assistant

Answers questions one at a time. Does not fill a table.

B Information Extraction, then ai_extract

Design on samples in the UI, then scale the same engine in SQL.

C Supervisor Agent per field

Coordination machinery for a job with nothing to coordinate.

D ai_summarize

A summary is prose. The table needs three separate fields.

THE SIGNAL

Same three fields from every contract, into a table, designed on samples then run nightly. That is the UI-then-SQL pattern.

> An insurer's underwriting agent must follow a fixed, proprietary sequence of checks, run in-house scoring code between steps, and call a pricing API with custom retry logic.
>
> The team tried a Supervisor Agent, but it could not express the fixed branching between steps.

What should the engineer do?

A Describe the steps in a Knowledge Assistant's instructions

B Give the Supervisor Agent more expert feedback until it follows the sequence

C Use Information Extraction to pull the underwriting fields

D Build a custom agent in code, wrapped as a ResponsesAgent

◆ PAUSE AND DECIDE

What should the engineer do?

A Knowledge Assistant instructions

A document Q and A builder cannot run scoring code or call APIs.

B More Supervisor feedback

Feedback tunes behaviour. It cannot guarantee a fixed branch.

C Information Extraction

Pulls fields. Does not run a sequence or call a pricing API.

D Custom agent in code

Fixed branching, proprietary code and custom retries need code.

THE SIGNAL

Could not express the fixed branching. A stated need the builders cannot meet is exactly when code is right.

What to carry forward

- **Knowledge Assistant:** cited Q and A over documents, tuned by experts without code.
- **Supervisor Agent:** one endpoint coordinating Genie Agents, assistants, functions and MCP.
- **Information Extraction:** design fields in the UI, run at scale with ai_extract.
- Walk the list in order. Write code only for a stated need no builder meets.
- Genie reads tables, not PDFs. Supervisor API is not Supervisor Agent.

> ON THE EXAM
>
> That completes section 1. Next, section 2: module 06 on choosing the right source documents.